

LC 209: Minimum size subarray sum

COMPANIES: DE SHAW, DROPBOX, SAP

Problem Statement

There will be given: an **array**, and a **target**.

Need a window within array such that $\text{sum} \geq \text{target}$.

O/P: **minimum window size**

Example Walkthrough

target = 7



So **[4, 3]** has $\text{sum} = 7$ and $\text{size} = 2$.

Hence, **O/P = 2**



Handwritten notes by Arpit Raj

Reference: Original Handwritten Notes

Approach: Sliding Window

- 1 we fix l to 0
- 2 r can vary from 0 to n
- 3 while ($\text{window sum} \geq \text{target}$) we move l closer to r , $\text{window sum} -= \text{nums}[l]$
- 4 update window size

C++ Implementation

```
class Solution { public: int minSubArrayLen(int target, vector<int>& nums)
{ int n=nums.size(); int l=0; int currsum=0; int minuwid=INT_MAX; for(int
r=0;r<n;r++){ currsum+=nums[r]; while(currsum>=target){
minuwid=min(minuwid,r-l+1); currsum-=nums[l]; l++; } }
if(minuwid==INT_MAX) return 0; else return minuwid; } };
```